

CS432: Databases

Structured Query Language (SQL)

Instructor
Yogesh K. Meena

Lecture no.
9

CSE, IIT Gandhinagar
February 04, 2026

Queries on a Multiple Relations



Queries on a Multiple Relations

```
select  $A_1, A_2, \dots, A_n$   
from  $r_1, r_2, \dots, r_m$   
where  $P$ 
```



Cartesian product of the relations

for each tuple t_1 **in** relation r_1
 for each tuple t_2 **in** relation r_2
 ...
 for each tuple t_m **in** relation r_m
 Concatenate $t_1, t_2, \dots, t_m$ into a single tuple t
 Add t into the result relation

Cartesian product of the relations

- Schema

*(instructor.ID, instructor.name, instructor.dept_name, instructor.salary
teaches.ID, teaches.course_id, teaches.sec_id, teaches.semester, teaches.year)*

- A more simplified version

Cartesian product of the relations

- Schema

*(instructor.ID, instructor.name, instructor.dept_name, instructor.salary
teaches.ID, teaches.course_id, teaches.sec_id, teaches.semester, teaches.year)*

- A more simplified version

*(instructor.ID, name, dept_name, salary
teaches.ID, course_id, sec_id, semester, year)*

Cartesian product

instructor

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

teaches

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

inst.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Comp. Sci.	65000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	12121	FIN-201	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Finance	90000	10101	CS-101	1	Fall	2009
12121	Wu	Finance	90000	10101	CS-315	1	Spring	2010
12121	Wu	Finance	90000	10101	CS-347	1	Fall	2009
12121	Wu	Finance	90000	12121	FIN-201	1	Spring	2010
12121	Wu	Finance	90000	15151	MU-199	1	Spring	2010
12121	Wu	Finance	90000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

The Natural Join

Natural join considers only those pairs of tuples with the same value on those attributes that appear in the schemas of both relations

- Example:

The Natural Join

Natural join considers only those pairs of tuples with the same value on those attributes that appear in the schemas of both relations

- Example:

ID	name	dept-name	salary	course-id	sec-id	semester	year
10101	Srinivasan	Comp. Sci.	65000	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000	CS-347	1	Fall	2009
12121	Wu	Finance	90000	FIN-201	1	Spring	2010
15151	Mozart	Music	40000	MU-199	1	Spring	2010
22222	Einstein	Physics	95000	PHY-101	1	Fall	2009
32343	El Said	History	60000	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000	CS-319	1	Spring	2010
76766	Crick	Biology	72000	BIO-101	1	Summer	2009
76766	Crick	Biology	72000	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000	EE-181	1	Spring	2009

The Natural Join

Natural join considers only those pairs of tuples with the same value on those attributes that appear in the schemas of both relations

- No repetition of attributes that appear in the schemas of both relations.
- A pre-defined order
 - Common attributes to the schemas of both relations
 - Attributes unique to the schema of the first relation
 - Attributes unique to the schema of the second relation

The Natural Join

For all instructors in the university who have taught some course, find their names and the course ID of all courses they taught

The Natural Join

For all instructors in the university who have taught some course, find their names and the course ID of all courses they taught

Before:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```

After:

The Natural Join

For all instructors in the university who have taught some course, find their names and the course ID of all courses they taught

Before:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```

After:

```
select name, course_id
from instructor natural join teaches;
```

The Natural Join

In general:

select $A_1, A_2, \dots, A_n$

from r_1 natural join r_2 natural join $\dots$, natural join r_m

where P

The Natural Join

In general:

```
select  $A_1, A_2, \dots, A_n$   
from  $r_1$  natural join  $r_2$  natural join ... natural join  $r_m$   
where  $P$ 
```

From clause in general:

```
from  $E_1, E_2, \dots, E_n$   
 $E_i$  can be a single relation or an expression involving natural joins.
```

The Natural Join

List the names of instructors along with the the titles of courses that they teach.

Instructor: (ID, name, dept_name, salary)

Teaches: (ID, course_id, sec_id)

Course: (course_id, title, dept_name, credits)



The Natural Join

List the names of instructors along with the the titles of courses that they teach.

Instructor: (ID, name, dept_name, salary)

Teaches: (ID, course_id, sec_id)

Course: (course_id, title, dept_name, credits)

Query option 1:

select name, title

from instructor **natural join** teaches, course

where teaches.course_id= course.course_id;

The Natural Join

List the names of instructors along with the the titles of courses that they teach.

Instructor: (ID, name, dept_name, salary)

Teaches: (ID, course_id, sec_id)

Course: (course_id, title, dept_name, credits)

Query option 1:

```
select name, title
from instructor natural join teaches, course
where teaches.course_id= course.course_id;
```

Query option 2:

```
select name, title
from instructor natural join teaches natural join course
```

Which one works correctly?

The Natural Join

List the names of instructors along with the the titles of courses that they teach.

Instructor: (ID, name, dept_name, salary)

Teaches: (ID, course_id, sec_id)

Course: (course_id, title, dept_name, credits)

A possible solution:

```
select name, title
```

```
from (instructor natural join teaches) join course using (course_id);
```

Additional Basic Operations

- Rename
- String
- Attribute specification in select clause
- Ordering the display of tuples
- Where clause predicates



Additional Basic Operations

A sample query:

```
select name, course_id  
from instructor, teaches  
where instructor.ID = teaches.ID;
```

Result relation with output attributes: name, course_id

Potential issues?

Additional Basic Operations

A sample query:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```

Result relation with output attributes: name, course_id

Potential issues?

- Attribute name may get duplicated
- Arithmetic expression in the select clause
- We just wish to change the attribute name

Rename Operation

- The SQL allows renaming relations and attributes using the **as** clause: *old-name as new-name*

A sample query:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```

Rename Operation

- The SQL allows renaming relations and attributes using the **as** clause: *old-name as new-name*

A sample query:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```

The Rename Operation:

```
select name as instructor_name, course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```


Rename Operation

For all instructors in the university who have taught some course, find their names and the course ID of all courses they taught.

Shortening by the rename operation:

Rename Operation

For all instructors in the university who have taught some course, find their names and the course ID of all courses they taught.

Shortening by the rename operation:

```
select T.name, S.course_id  
from instructor as T, teaches as S  
where T.ID= S.ID;
```

Rename Operation

Find the names of all instructors whose salary is greater than at least one instructor in the Biology department.

Compare tuples in the same relation using rename operation:

Rename Operation

Find the names of all instructors whose salary is greater than at least one instructor in the Biology department.

Compare tuples in the same relation using rename operation:

```
select distinct T.name  
from instructor as T, instructor as S  
where T.salary > S.salary and S.deptname = 'Biology';
```

Table alias, or a correlation variable, or a tuple variable.

Keyword **as** is optional and may be omitted: instructor as T
 $\equiv$ instructor T

String Operations

- Case sensitive as per SQL standards: `comp.sci.` $\neq$ `Comp.Sci.`
- Supports concatenation, substring extraction, finding length, case conversion, space removal at the end of the string and so on.
- Pattern matching with two symbols (`%` and `_`). `'Intro%'` matches any string beginning with `'Intro'`. `'%Comp%'` matches any string containing `'Comp'` as a substring. `'_ _ _ %'` matches any string of at least three. `'_ _ _'` matches any string of exactly three characters.
- **like operator**
`select dept`
`from department`
`where building like '%Watson%';`

Ordering the Display of Tuples

- List in alphabetic order the names of all instructors
select distinct name
from instructor
order by name;
- We may specify **desc** for descending order or **asc** for ascending order, for each attribute; ascending order is the default.
Example: **order by name desc**
- Can sort on multiple attributes?

Ordering the Display of Tuples

- List in alphabetic order the names of all instructors
select distinct name
from instructor
order by name;
- We may specify **desc** for descending order or **asc** for ascending order, for each attribute; ascending order is the default.
Example: **order by name desc**
- Can sort on multiple attributes?
Example: **order by dept_name, name**

Where Clause Predicates

SQL includes a **between** comparison operator.

Example: Find the names of all instructors with salary between \$90,000 and \$100,000 (that is $\geq$ \$90,000 and $\leq$ \$100,000)

Where Clause Predicates

SQL includes a **between** comparison operator.

Example: Find the names of all instructors with salary between \$90,000 and \$100,000 (that is $\geq$ \$90,000 and $\leq$ \$100,000)

```
select name  
from instructor  
where salary between 90000 and 100000;
```

Where Clause Predicates

SQL includes a **between** comparison operator.

Example: Find the names of all instructors with salary between \$90,000 and \$100,000 (that is $\geq \$90,000$ and $\leq \$100,000$)

```
select name  
from instructor  
where salary between 90000 and 100000;
```

Example: Find the instructor names and courses they taught for all instructors in the Biology department who have taught some course

Where Clause Predicates

SQL includes a **between** comparison operator.

Example: Find the names of all instructors with salary between \$90,000 and \$100,000 (that is $\geq$ \$90,000 and $\leq$ \$100,000)

```
select name  
from instructor  
where salary between 90000 and 100000;
```

Example: Find the instructor names and courses they taught for all instructors in the Biology department who have taught some course

```
select name, course_id  
from instructor, teaches  
where instructor.ID = teaches.ID and dept_name = 'Biology';
```

Where Clause Predicates

```
select name, course_id
from instructor, teaches
where (instructor.ID, dept_name) = (teaches.ID, 'Biology');
```

Set Operations

Example: Find courses that ran in Fall 2024 or in Spring 2025.



Set Operations

Example: Find courses that ran in Fall 2024 or in Spring 2025.

(**select** course_id **from** section **where** sem = 'Fall' **and** year = 2024)
union

(**select** course_id **from** section **where** sem = 'Spring' **and** year = 2025)

Set Operations

Example: Find courses that ran in Fall 2024 or in Spring 2025.

(**select** course_id **from** section **where** sem = 'Fall' **and** year = 2024)
union

(**select** course_id **from** section **where** sem = 'Spring' **and** year = 2025)

Example: Find courses that ran in Fall 2024 and in Spring 2025.

Set Operations

Example: Find courses that ran in Fall 2024 or in Spring 2025.

```
(select course_id from section where sem = 'Fall' and year = 2024)  
union
```

```
(select course_id from section where sem = 'Spring' and year = 2025)
```

Example: Find courses that ran in Fall 2024 and in Spring 2025.

```
(select course_id from section where sem = 'Fall' and year = 2024)  
intersect
```

```
(select course_id from section where sem = 'Spring' and year = 2025)
```


Set Operations

Example: Find courses that ran in Fall 2024 or in Spring 2025.

```
(select course_id from section where sem = 'Fall' and year = 2024)  
union
```

```
(select course_id from section where sem = 'Spring' and year = 2025)
```

Example: Find courses that ran in Fall 2024 and in Spring 2025.

```
(select course_id from section where sem = 'Fall' and year = 2024)  
intersect
```

```
(select course_id from section where sem = 'Spring' and year = 2025)
```

Example: Find courses that ran in Fall 2024 but not in Spring 2025.

Set Operations

Example: Find courses that ran in Fall 2024 or in Spring 2025.

```
(select course_id from section where sem = 'Fall' and year = 2024)  
union
```

```
(select course_id from section where sem = 'Spring' and year = 2025)
```

Example: Find courses that ran in Fall 2024 and in Spring 2025.

```
(select course_id from section where sem = 'Fall' and year = 2024)  
intersect
```

```
(select course_id from section where sem = 'Spring' and year = 2025)
```

Example: Find courses that ran in Fall 2024 but not in Spring 2025.

```
(select course_id from section where sem = 'Fall' and year = 2024)  
except
```

```
(select course_id from section where sem = 'Spring' and year = 2025)
```

Set Operations

- Each of the set operations (**union**, **intersect**, and **except**) automatically eliminates duplicates.
- To retain all duplicates use the: **union all**, **intersect all**, **except all**.

Null Values

- **null** signifies an unknown value or that a value does not exist.
- The result of any arithmetic expression involving **null** is **null**.
- The predicate **is null** can be used to check for null values.

Example: Find all instructors whose salary is null.

Null Values

- **null** signifies an unknown value or that a value does not exist.
- The result of any arithmetic expression involving **null** is **null**.
- The predicate **is null** can be used to check for null values.

Example: Find all instructors whose salary is null.

```
select name  
from instructor  
where salary is null;
```

Null Values

- **null** signifies an unknown value or that a value does not exist.
- The result of any arithmetic expression involving **null** is **null**.
- The predicate **is null** can be used to check for null values.

Example: Find all instructors whose salary is null.

```
select name  
from instructor  
where salary is null;
```

- The predicate in a **where** clause can involve Boolean operations (and, or, not).
 - **and**: The result of true and unknown is unknown, false and unknown is false, while unknown and unknown is unknown.
 - **or**: The result of true or unknown is true, false or unknown is unknown, while unknown or unknown is unknown.
 - **not**: The result of not unknown is unknown.

Aggregate Functions

- These functions operate on the multiset of values of a column of a relation, and return a value.
 - **avg**: average value
 - **min**: minimum value
 - **max**: maximum value
 - **sum**: sum of values
 - **count**: number of values
- **Example**: Find the average salary of instructors in the Computer Science department.

Aggregate Functions

- These functions operate on the multiset of values of a column of a relation, and return a value.
 - **avg**: average value
 - **min**: minimum value
 - **max**: maximum value
 - **sum**: sum of values
 - **count**: number of values
- **Example**: Find the average salary of instructors in the Computer Science department.

```
select avg (salary)
from instructor
where dept_name= 'Comp. Sci.';
```


Aggregate Functions: Group By

- Find the average salary of instructors in each department

```
select dept_name avg (salary) as avg_salary  
from instructor  
group by dept_name;
```

ID	name	dept_name	salary
76766	Crick	Biology	72000
45565	Katz	Comp. Sci.	75000
10101	Srinivasan	Comp. Sci.	65000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000
12121	Wu	Finance	90000
76543	Singh	Finance	80000
32343	El Said	History	60000
58583	Califieri	History	62000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
22222	Einstein	Physics	95000

dept_name	avg_salary
Biology	72000
Comp. Sci.	77333
Elec. Eng.	80000
Finance	85000
History	61000
Music	40000
Physics	91000

Aggregate Functions: Having Clause

- Find the names and average salaries of all departments whose average salary is greater than 42000

Aggregate Functions: Having Clause

- Find the names and average salaries of all departments whose average salary is greater than 42000

```
select dept_name avg (salary) as avg_salary  
from instructor  
group by dept_name;  
having avg (salary) > 42000;
```

Aggregate Functions: Having Clause

- Find the names and average salaries of all departments whose average salary is greater than 42000

```
select dept_name avg (salary) as avg_salary  
from instructor  
group by dept_name;  
having avg (salary) > 42000;
```

- Note:** predicates in the **having** clause are applied after the formation of groups whereas predicates in the **where** clause are applied before forming groups.

Nested Subqueries

- SQL provides a mechanism for the nesting of subqueries. A subquery is a select-from-where expression that is nested within another query.
- The nesting can be done in the following SQL query

```
select  $A_1, A_2, \dots, A_n$   
from  $r_1, r_2, \dots, r_m$   
where  $P$ 
```

- as follows:
 - **From clause:** r_i can be replaced by any valid subquery
 - **Where clause:** P can be replaced with an expression of the form: $B < operation > (subquery)$. B is an attribute and $< operation >$ to be defined later.
 - **Select clause:** A_i can be replaced by a subquery that generates a single value.

Nested Subqueries

Previous example (set operations): Find courses that ran in Fall 2024 and in Spring 2025.

```
(select course_id from section where sem = 'Fall' and year = 2024)
intersect
(select course_id from section where sem = 'Spring' and year = 2025)
```

Set Membership: Find all the courses taught in the both the Fall 2009 and Spring 2010 semesters.

Nested Subqueries

Previous example (set operations): Find courses that ran in Fall 2024 and in Spring 2025.

```
(select course_id from section where sem = 'Fall' and year = 2024)
intersect
(select course_id from section where sem = 'Spring' and year = 2025)
```

Set Membership: Find all the courses taught in the both the Fall 2009 and Spring 2010 semesters.

```
select course_id
from section
where semester = 'Fall' and year = 2009 and
      course_id in (select course_id
                    from section
                    where semester = 'Spring' and year = 2010);
```

Nested Subqueries

Not in: Find all the courses taught in the the Fall 2009 but not in Spring 2010 semesters.

Nested Subqueries

Not in: Find all the courses taught in the the Fall 2009 but not in Spring 2010 semesters.

```
select course_id
from section
where semester = 'Fall' and year = 2009 and
      course_id not in (select course_id
                        from section
                        where semester = 'Spring' and year = 2010);
```

Nested Subqueries

Previous example (set comparison): Find the names of all instructors whose salary is greater than at least one instructor in the Biology department.

Nested Subqueries

Previous example (set comparison): Find the names of all instructors whose salary is greater than at least one instructor in the Biology department.

```
select distinct T.name  
from instructor as T, instructor as S  
where T.salary > S.salary and S.deptname = 'Biology';
```

Nested Subqueries

Previous example (set comparison): Find the names of all instructors whose salary is greater than at least one instructor in the Biology department.

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary and S.deptname = 'Biology';
```

Set Comparison

```
select name
from instructor
where salary > some (select salary
                      from instructor
                      where dept_name = 'Biology');
```

Several variants of 'some' are available: < some, <= some, >= some, = some, and <> some

Nested Subqueries

Set Comparison: Find the names of all instructors that have a salary value greater than that of each instructor in the Biology department

Nested Subqueries

Set Comparison: Find the names of all instructors that have a salary value greater than that of each instructor in the Biology department

```
select name from instructor
where salary > all (select salary
                    from instructor
                    where dept_name = 'Biology');
```

Nested Subqueries

Set Comparison: Find the names of all instructors that have a salary value greater than that of each instructor in the Biology department

```
select name from instructor
where salary > all (select salary
                    from instructor
                    where dept_name = 'Biology');
```

Test for Empty Relations Find all the courses taught in the both the Fall 2009 and Spring 2010 semesters.

Nested Subqueries

Set Comparison: Find the names of all instructors that have a salary value greater than that of each instructor in the Biology department

```
select name from instructor
where salary > all (select salary
                    from instructor
                    where dept_name = 'Biology');
```

Test for Empty Relations Find all the courses taught in the both the Fall 2009 and Spring 2010 semesters.

```
select course_id
from section as S
where semester = 'Fall' and year= 2009 and
      exists (select*
            from section as T
            where sem = 'Spring' and year= 2010 and S.course_id =
T.course_id);
```


Modification of the Database

- Covered in Hands-on and practice exercises in previous lecture.
- Deletion of tuples from a given relation.
- Insertion of new tuples into a given relation
- Updating of values in some tuples in a given relation

Deletion

`delete from  $r$`
`where  $p$ ;`

Modification of the Database

- Covered in Hands-on and practice exercises in previous lecture.
- Deletion of tuples from a given relation.
- Insertion of new tuples into a given relation
- Updating of values in some tuples in a given relation

Deletion

delete from r
where p ;

Insertion: Ordering matters!

insert into course
values ('CS-437', 'Databases', 'Comp. Sci.', 4);

Modification of the Database

— Covered in Hands-on and practice exercises in previous lecture.

- Deletion of tuples from a given relation.
- Insertion of new tuples into a given relation
- Updating of values in some tuples in a given relation

Deletion

delete from *r*
where *p*;

Insertion: Ordering matters!

insert into course
values ('CS-437', 'Databases', 'Comp. Sci.', 4);

Insertion: Ordering does not matters!

insert into course (title, course_id, credits, dept_name)
values ('Databases', 'CS-437', 4, 'Comp. Sci.');

Modification of the Database

Example – Ordering of update matters! Let us suppose that all instructors with salary over \$100,000 receive a 3 percent raise, whereas all other received a 5 percent raise.

Modification of the Database

Example – Ordering of update matters! Let us suppose that all instructors with salary over \$100,000 receive a 3 percent raise, whereas all other received a 5 percent raise.

update instructors

set salary = salary*1.03

where salary > 100000;

Modification of the Database

Example – Ordering of update matters! Let us suppose that all instructors with salary over \$100,000 receive a 3 percent raise, whereas all other received a 5 percent raise.

update instructors

set salary = salary*1.03

where salary > 100000;

update instructors

set salary = salary*1.05

where salary > 100000;

Modification of the Database

Example – Ordering of update matters! Let us suppose that all instructors with salary over \$100,000 receive a 3 percent raise, whereas all other received a 5 percent raise.

```
update instructors  
set salary = salary*1.03  
where salary > 100000;
```

```
update instructors  
set salary = salary*1.05  
where salary > 100000;
```

```
update instructors  
set salary = case  
                when salary <= 100000 then salary*1.05  
                else salary*1.03  
end
```

Modification of the Database

```
case
  when  $pred_1$  then  $result_1$ 
  when  $pred_2$  then  $result_2$ 
  ...
  when  $pred_n$  then  $result_n$ 
  else  $result_0$ 
end
```


Acknowledgments/Contributions

- Some of the images utilized in these slides are subject to copyright - Abraham Silberschatz, Henry Korth, and S. Sudarshan. Database System Concepts. 6th Edition, McGraw-Hill Education
- Contributor 2024-25, 2025-26 - Yogesh K. Meena, IIT Gandhinagar